

Notebook-PLA2

July 16, 2019

M2.956 ù Anàlisi de sentiments i xarxes socials
Màster universitari de Ciència de Dades (Data Science)
Estudis d'Informàtica, Multimèdia i Telecomunicació

1 Mòdul 2: Extracció de sentiments i opinions

1.1 Scripts dels mètodes explicats en el mòdul

A continuació, es presenten els scripts que exemplifiquen els mètodes explicats a PLA-2.

Paquets requerits per a executar aquest notebook, a més dels comentats en el notebook anterior (Notebook-PLA1): - rdflib i neuralcoref (després d'instal·lar spaCy). Per a la instal·lació podeu usar "pip" o "conda", segons el sistema que tingueu instal·lat.

- Recordeu que heu de mantenir la mateixa estructura de les carpetes perquè el codi trobi els fitxers .csv i les imatges que s'usen.

Nota: Es recomana emprar la versió 3.6 de Python.

1.2 1. Detecció dels components d'una opinió

1.2.1 1.1 Detecció de les diferents denominacions del target

Mostrem com es detecta el target en les seves diferents variacions:

Hiperònim
Sinònim
Referència anafòrica

Comprovar si el terme A és un hiperònim del terme B consultant DBpedia

```
In [1]: #Importem la llibreria per a poder fer les consultes a la DBpedia.  
        #La resposta de DBpedia és un graf i la informació s'estructura en format RDF.  
import rdflib  
from rdflib import Graph, URIRef, RDF  
  
#Mètode per a saber si A és un hiperònim de B.  
def is_hyperonym(source,target):  
    dbpedia_uri = URIRef('http://dbpedia.org/resource/'+ source) #Uri que apunta a A a
```

```

g = Graph()
g.parse(dbpedia_uri) #S'analitza la resposta de DBpedia en forma de graf
url_matches = []
for obj in g.objects(subject=dbpedia_uri, predicate=RDF.type): #Busquem en el graf
    #sobre els objectes del predicat 'type' quan el subjecte és la uri que apunta
    #Es recullen les referències a l'objecte de 'type' segons Yago en les quals hi
    if 'http://dbpedia.org/class/yago/' in obj and target in obj:
        url_matches.append(obj)
    #Si hi ha referències a l'objecte de type que contenen B, llavors A és el seu hiperònim
if len(url_matches) > 0:
    print(target, "ES UN HIPERÓNIMO DE", source)
else:
    print(target, "NO ES UN HIPERÓNIMO DE ", source)

is_hyperonym('IPhone', 'Telephone')

```

Telephone ES UN HIPERÓNIMO DE IPhone

Comprovar si A és un sinònim de B amb word embeddings.

```

In [2]: import gensim
import nltk
from nltk.corpus import wordnet as wn

#Carreguem el model Word2Vec d'opinions sobre smartphones
model = gensim.models.Word2Vec.load('MODEL-MOBILEPHONE-REVIEWS-PL2')

term = 'samsung'

#Obtenim els phrases més similars al terme a partir del model Word2Vec
most_similar = model.wv.most_similar(positive=term)

print("PHRASES CON VALOR DE SIMILITUD", most_similar)

#De la llista de phrases més similars (score de similitud > 0.4) prenem els sintagmes
#(NN, NS o JJ+NN, NN+NN, NN+NS).

def is_np(candidate):
    test = False
    tokens = candidate.split()
    tagged_tokens = nltk.pos_tag(tokens)
    #Si és un terme multiparaula
    if len(tagged_tokens) > 1:
        PoS_initial = tagged_tokens[0][1][:2]
        PoS_final = tagged_tokens[-1][1][:2]
        if ((PoS_initial == 'NN' or PoS_initial == 'JJ') and (PoS_final == 'NN' or PoS_
            test = True

```

```

        #Si és un terme monoparaula
    else:
        if len(candidate) > 1 and tagged_tokens[0][1][:2] == 'NN' or tagged_tokens[0][1] == 'NNP':
            test = True
        return test

target_candidates = [tc[0] for tc in most_similars if tc[1] > 0.4 and is_np(tc[0]) == True]

print("CANDIDATOS A TARGET", target_candidates)

PHRASES CON VALOR DE SIMILITUD [('samsung products', 0.5297492146492004), ('htc', 0.5151442885144285)]
CANDIDATOS A TARGET ['samsung products', 'htc', 'lg', 'galaxy', 'touch wiz', 'touchwiz', 'samsung']

```

Referència anafòrica al target

```

In [3]: import neuralcoref
import spacy
import pytorch

nlp = spacy.load('en_coref_sm')
#o import en_coref_sm nlp = en_coref_sm.load()
#o en_coref_lg per a un model entrenat amb un corpus més gran

doc = nlp(u'I bought an iPhone a few days ago. It was such a nice phone. \
The touch screen was really cool. The voice quality was clear too. \
However, my mother was mad with me as I did not tell her before I bought it. \
Although the battery life was not long, that is ok for me')

doc._.coref_resolved

/Library/Frameworks/Python.framework/Versions/3.6/lib/python3.6/importlib/_bootstrap.py:219: RuntimeWarning:
return f(*args, **kwargs)

```

ModuleNotFoundError

Traceback (most recent call last)

```

<ipython-input-3-673a6fbae1d2> in <module>
      1 import neuralcoref
      2 import spacy
----> 3 import pytorch
      4
      5 nlp = spacy.load('en_coref_sm')

```

ModuleNotFoundError: No module named 'pytorch'

1.2.2 1.2 Detecció dels aspectes

Obtenir les propietats d'un producte segons una infobox de la Wikipedia

In [4]: `import requests`

*#Obtenir les propietats del target 'IPHONE', consultant el graf de DBpedia
#i obtenint la informació en format RDF*

```
def get_properties(target):
    data = requests.get("http://dbpedia.org/data/" + target + ".json").json()
    uris = data["http://dbpedia.org/resource/" + target]
    properties = []

    for u in uris:
        if '/property/' in u:
            us = u.split('/')
            properties.append(us[-1])
    return properties
```

`target = 'iPhone'`

`properties_target = get_properties(target)`

`print(properties_target)`

`['unitssold', 'developer', 'reason', 'sound', 'date', 'dimensions', 'power', 'caption', 'displ`

Detecció dels aspectes per a clustering dels seus synsets de Wordnet

In [5]: *#D'una opinió, prenem els candidats a ser aspectes del target*

```
import nltk
from nltk import word_tokenize
from nltk.util import ngrams
from nltk.collocations import *
import re
import collections
```

#Un candidat a ser aspecte és un terme monoparaula o multiparaula que forma un sintagma

```
def is_np(candidate):
    test = False
    tokens = candidate.split()
    tagged_tokens = nltk.pos_tag(tokens)
    if len(tagged_tokens) > 1:
        PoS_initial = tagged_tokens[0][1][:2]
        PoS_final = tagged_tokens[-1][1][:2]
```

```

        if ((PoS_initial == 'NN' or PoS_initial == 'JJ') and (PoS_final == 'NN' or PoS_
            test = True
    else:
        if len(candidate) > 1 and tagged_tokens[0][1][:2] == 'NN' or tagged_tokens[0][
            test = True
    return test

def get_np_candidates(text):
    #Definim les mètriques que avaluaran si un n-grama pot ser una collocation
    bigram_measures = nltk.collocations.BigramAssocMeasures()
    #Tokenitzem i obtenim els tokens del text.
    tokens = [w for w in word_tokenize(text.lower())]
    #Cerca de bigrames
    bigramfinder = BigramCollocationFinder.from_words(tokens)
    #Es prenen els bigrames que no tenen signes de puntuació, etc.
    bigramfinder.apply_word_filter(lambda w: (re.match(r'\W', w)))
    #N millors candidats a ser col·locacions, després de passar pel càlcul del PMI.
    bigram_candidates = bigramfinder.nbest(bigram_measures.pmi,100)
    #Transformació de la tupla del bigrama a collocation
    collocation_candidates = [" ".join(bc) for bc in bigram_candidates]
    #Triem els candidats que són sintagmes nominals.
    np_candidates = [c for c in tokens + collocation_candidates if is_np(c) == True]
    return np_candidates

```

opinion = "I bought an iPhone a few days ago. It was such a nice phone. The touch screen was great. The voice quality was clear too. Although the battery life was not long, that is ok for me. My mother was mad with me as I did not tell her before I bought it. She also thought that I was stupid and wanted me to return it to the shop"

```
np_candidates = get_np_candidates(opinion)
```

In [6]: *#Transformem els candidats a aspecte als seus synsets de Wordnet, quan aquests synsets*

```

from nltk.corpus import wordnet as wn
from nltk.stem.wordnet import WordNetLemmatizer

lem = WordNetLemmatizer()

np_candidates.append('telephone') #Posem l'hiperònim de iPhone

candidates_synsets_list = []

for npc in np_candidates:
    candidates_synsets = wn.synsets(lem.lemmatize(npc.replace(' ', '_')), pos=wn.NOUN)
    for cs in candidates_synsets:
        if '"' + npc.replace(' ', '_') + ".n" in str(cs):
            candidates_synsets_list.append(cs)

```

In [7]: *#Transformem les propietats del target, segons Wikipedia, als seus synsets de Wordnet,*

```

import requests

#Obtenir les propietats del target 'iPhone', segons Wikipedia

def get_properties(target):
    data = requests.get("http://dbpedia.org/data/" + target + ".json").json()
    uris = data["http://dbpedia.org/resource/" + target]
    properties = []

    for u in uris:
        if '/property/' in u:
            us = u.split('/')
            properties.append(us[-1])
    return properties

target = 'iPhone'

properties_target = get_properties(target)

features_synsets_list = []

for pt in properties_target:
    features_synsets = wn.synsets(lem.lemmatize(pt.replace(' ', '_')), pos=wn.NOUN)
    for fs in features_synsets:
        if "" + pt + ".n" in str(fs):
            features_synsets_list.append(fs)

In [8]: #Creem els vectors que representen als candidats a aspecte.
#Els vectors es construeixen calculant la distància de cada synset del candidat a aspecte
#la propietat del target.

from numpy import matrix

def create_vector(synset, synsets_vocabulary):
    vector = [s.wup_similarity(synset) for s in synsets_vocabulary] #Wu and Palmer score
    return vector

vectors = [create_vector(v, features_synsets_list) for v in candidates_synsets_list]

X = matrix(vectors)

print(X)

[[0.16666667 0.13333333 0.33333333 ... 0.30769231 0.23529412 0.26666667]
 [0.15384615 0.125      0.46153846 ... 0.57142857 0.44444444 0.5       ]
 [0.125      0.10526316 0.375      ... 0.35294118 0.28571429 0.31578947]
 ...

```

```
[0.38095238 0.57142857 0.11111111 ... 0.10526316 0.08695652 0.0952381 ]
[0.44444444 0.77777778 0.13333333 ... 0.125         0.1         0.11111111]
[0.44444444 0.66666667 0.13333333 ... 0.125         0.1         0.11111111]]
```

```
In [9]: from sklearn.cluster import KMeans
```

```
#Clustering dels vectors
```

```
num_clusters = 5
```

```
km = KMeans(n_clusters=num_clusters, n_init=10) # n_init per a mantenir la consistència
```

```
km.fit(X)
```

```
labels = km.labels_.tolist()
```

```
labels_color_map = {
    0: '#20b2aa', 1: '#ff7373', 2: '#ffe4e1', 3: '#005073', 4: '#4d0404'
}
```

```
class_0 = []
```

```
class_1 = []
```

```
class_2 = []
```

```
class_3 = []
```

```
class_4 = []
```

```
for i in range(len(labels)):
    if labels[i] == 0:
        class_0.append(candidates_synsets_list[i])
    if labels[i] == 1:
        class_1.append(candidates_synsets_list[i])
    if labels[i] == 2:
        class_2.append(candidates_synsets_list[i])
    if labels[i] == 3:
        class_3.append(candidates_synsets_list[i])
    if labels[i] == 4:
        class_4.append(candidates_synsets_list[i])
```

```
print("CLASS_0", class_0)
```

```
print("CLASS_1", class_1)
```

```
print("CLASS_2", class_2)
```

```
print("CLASS_3", class_3)
```

```
print("CLASS_4", class_4)
```

```
CLASS_0 [Synset('touch.n.01'), Synset('touch.n.05'), Synset('touch.n.08'), Synset('touch.n.09')
CLASS_1 [Synset('voice.n.07'), Synset('life.n.08'), Synset('life.n.10'), Synset('life.n.11'), S
CLASS_2 [Synset('phone.n.02'), Synset('touch.n.03'), Synset('touch.n.04'), Synset('touch.n.06')]
```

```

CLASS_3 [Synset('touch.n.02'), Synset('touch.n.10'), Synset('quality.n.03'), Synset('life.n.02')
CLASS_4 [Synset('screen.n.01'), Synset('screen.n.03'), Synset('screen.n.04'), Synset('screen.n.

```

In [11]: *#Visualització*

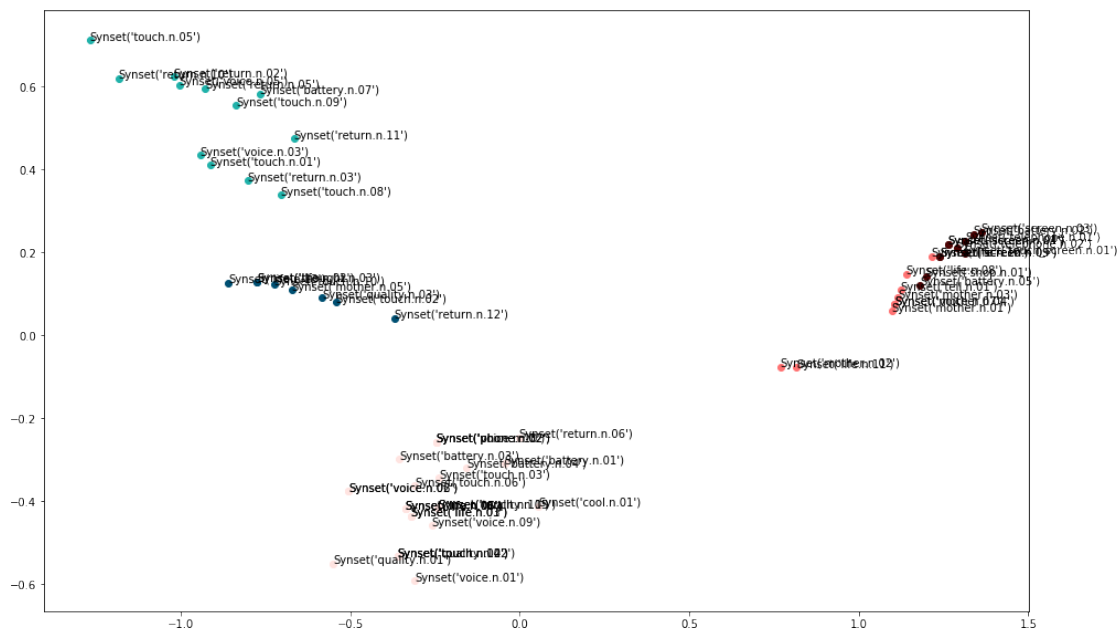
```

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

reduced_data = PCA(n_components=2).fit_transform(X)

fig, ax = plt.subplots(figsize=(16,10))
for index, instance in enumerate(reduced_data):
    pca_comp_1, pca_comp_2 = reduced_data[index]
    color = labels_color_map[labels[index]]
    ax.scatter(pca_comp_1, pca_comp_2, c=color)
    ax.annotate(s=str(candidates_synsets_list[index]),xy=(pca_comp_1, pca_comp_2))
plt.show()

```



Detecció dels aspectes a partir d'opinion words

In [12]: *#Obtenim els candidats a ser aspectes i la llista d'opinion words*

```

import gensim
from nltk.stem.wordnet import WordNetLemmatizer

lem = WordNetLemmatizer()

```



```

#Els candidats a aspecte ja han estat obtinguts abans (np_candidates).
#Només cal lematitzar-los.

print(np_candidates)

#Creem una llista d'opinion words a partir del diccionari AFINN
opinion_words_file = 'AFINN-111.txt'

opinion_words = []

with open(opinion_words_file, 'r') as of:
    ol = of.readlines()
    for wo in ol:
        w, o = wo.strip().split('\t')
        opinion_words.append(w)

#Obtenim els lemes unificats dels candidats a aspecte si no són a la llista d'opinion
def unify(l):
    term_unified = l
    if l == 'touch_screen':
        term_unified = 'touchscreen'
    return term_unified

aspect_candidates = [unify(lem.lemmatize(ac.replace(' ', '_')) for ac in np_candidates)

print(aspect_candidates)

['bought', 'iphone', 'days', 'phone', 'touch', 'screen', 'cool', 'voice', 'quality', 'battery',
['bought', 'iphone', 'day', 'phone', 'touch', 'screen', 'voice', 'quality', 'battery', 'life',

In [13]: #Carreguem el model d'opinions sobre smartphones

model= gensim.models.Word2Vec.load('MODEL-MOBILEPHONE-REVIEWS-PL2')

#Creem el vocabulari, que és la llista de candidats a terme i les opinion words que s
#i al model.

opinion_words_in_opinion = [ow for ow in word_tokenize(opinion.lower()) if ow in opin

vocabulary = aspect_candidates + opinion_words_in_opinion

vocabulary_in_model = [v.replace('_', ' ') for v in vocabulary if v.replace('_', ' ') in

#Creem els vectors dels elements del vocabulari, calculant la distància (cosine simil
#vector del terme del vocabulari segons el model d'opinions d'smartphones i els vector

```

```
#elements del vocabulari segons el mateix model.
```

```
import numpy as np
```

```
def sem_distance(a, o):  
    cosine_similarity = np.dot(model[a], model[o]) / (np.linalg.norm(model[a])* np.linalg.norm(model[o]))  
    return cosine_similarity
```

```
def create_vector(v, vocabulary):  
    vector = [sem_distance(v, vb) for vb in vocabulary]  
    return vector
```

```
vectors = [create_vector(v,vocabulary_in_model) for v in vocabulary_in_model]
```

```
/Library/Frameworks/Python.framework/Versions/3.6/lib/python3.6/site-packages/ipykernel_launcher.py
```

```
In [14]: #Clustering
```

```
from numpy import matrix  
from sklearn.cluster import KMeans  
from sklearn.decomposition import PCA  
import matplotlib.pyplot as plt
```

```
X = matrix(vectors)
```

```
num_clusters = 3
```

```
km = KMeans(n_clusters=num_clusters, n_init=5) # initial number in order to keep consistency
```

```
km.fit(X)
```

```
labels = km.labels_.tolist()
```

```
class_0 = []
```

```
class_1 = []
```

```
class_2 = []
```

```
for i in range(len(labels)):  
    if labels[i] == 0:  
        class_0.append(vocabulary_in_model[i])  
    if labels[i] == 1:  
        class_1.append(vocabulary_in_model[i])  
    if labels[i] == 2:  
        class_2.append(vocabulary_in_model[i])
```

```
print("CLASS_0", class_0)
```

```
print("CLASS_1", class_1)
print("CLASS_2", class_2)
```

```
CLASS_0 ['bought', 'mother', 'tell', 'bought', 'thought', 'return', 'shop', 'telephone', 'mad']
CLASS_1 ['iphone', 'touch', 'screen', 'voice', 'quality', 'ok', 'touchscreen', 'voice quality']
CLASS_2 ['day', 'phone', 'battery', 'life', 'phone', 'battery life']
```

1.2.3 1.3 Detecció de la polaritat

Preprocessament

In [15]: *#Preprocessament de tweets*

```
import re

## Treure emojis

text = "Adicto a la lectura. Apasionado de la política y la economía. Balonmano, natura
       Dibujo tebeos como croquetas, feminista y... a veces soy Clara te canta "

emoji_pattern = re.compile("[
    u"\U0001F600-\U0001F64F" # emoticons
    u"\U0001F300-\U0001F5FF" # symbols & pictographs
    u"\U0001F680-\U0001F6FF" # transport & map symbols
    u"\U0001F1E0-\U0001F1FF" # flags (iOS)
    "]" + flags=re.UNICODE)

print(text)
print(emoji_pattern.sub(r'', text)) # no emoji

## Treure urls

text = "AEP es un proyecto para difundir material anarquista existente. https://t.co/1

no_http_text = re.sub(r'(\s)http\S+', '', text) #\S+ matches any non-whitespace character

print (text)
print(no_http_text)

## Treure hashtags

text = "Tinc 4 #PremisTuitort per ser cuqui i fer #Rubiadas, dos @GatsMalvats"

print(text)

no_hash_arr_text = text = re.sub(r'#[@]', '', text)

print (no_hash_arr_text)
```

Adicto a la lectura. Apasionado de la política y la economía. Balonmano, naturaleza y mucha música.
Adicto a la lectura. Apasionado de la política y la economía. Balonmano, naturaleza y mucha música.
AEP es un proyecto para difundir material anarquista existente. <https://t.co/Nxv9arm4Ti> en consulta
AEP es un proyecto para difundir material anarquista existente. en consulta
Tinc 4 #PremisTuitort per ser cuqui i fer #Rubiadas, dos @GatsMalvats
Tinc 4 PremisTuitort per ser cuqui i fer Rubiadas, dos GatsMalvats

Classificació de la polaritat d'un twit

```
In [16]: import nltk
         from nltk import word_tokenize

         #Corpus d'entrenament

         #Tweets que es consideren de polaritat positiva

         pos_tweets = ['I love this car.', 'This view is amazing.', 'I feel great this morning',
                        'I am so excited about the concert.', 'He is my best friend.']

         #Tweets que es consideren de polaritat negativa

         neg_tweets = ['I do not like this car.', 'This view is horrible.', 'I feel tired this morning',
                        'I am not looking forward to the concert.', 'He is my enemy.']

         #Creem els documents d'entrenament: un per a cada tipus opinió

         docs = pos_tweets + neg_tweets

         print("TRAINING TWEETS",docs)

         #Definim el vectoritzador
         from sklearn.feature_extraction.text import TfidfVectorizer
         import numpy as np

         vectorizer = TfidfVectorizer(
             analyzer= 'word',
             )

         #Entrenament

         data_labels = []

         for i in range(len(pos_tweets)):
             data_labels.append('POS') #Data labelling

         for i in range(len(neg_tweets)):
             data_labels.append('NEG')
```

```

print("TRAINING DATA LABELS", data_labels)

X = vectorizer.fit_transform(docs)

X_train = X.toarray() #Matriu amb els vectors d'entrenament

from sklearn.naive_bayes import MultinomialNB

classifier = MultinomialNB() #Definició del classificador bayesià

bayes_model = classifier.fit(X=X_train, y=data_labels) #Entrenament del classificador
#el mètode al corpus d'entrenament

#Predicció

new_tweets = ['Larry is my friend', 'The film is horrible']

X1 = vectorizer.transform(new_tweets)

y_pred = bayes_model.predict(X1)

for i in range(len(new_tweets)):
    print ("LA POLARIDAD DE", new_tweets[i], "ES", y_pred[i])

```

```

TRAINING TWEETS ['I love this car.', 'This view is amazing.', 'I feel great this morning.', 'I
TRAINING DATA LABELS ['POS', 'POS', 'POS', 'POS', 'POS', 'NEG', 'NEG', 'NEG', 'NEG', 'NEG']
LA POLARIDAD DE Larry is my friend ES POS
LA POLARIDAD DE The film is horrible ES NEG

```

1.2.4 1.4 Opinion holder

Obtenir el perfil d'un compte

```

In [17]: import tweepy
from tweepy import API
from tweepy import OAuthHandler

CONSUMER_KEY = XXXXX
CONSUMER_SECRET = XXXXX
ACCESS_KEY = XXXXX
ACCESS_SECRET = XXXXX

auth = tweepy.OAuthHandler(CONSUMER_KEY, CONSUMER_SECRET)
auth.set_access_token(ACCESS_KEY, ACCESS_SECRET)

api = tweepy.API(auth)

```

```
description_user = api.get_user('@realDonaldTrump').description

print(description_user)
```

```
ModuleNotFoundError                                Traceback (most recent call last)
```

```
<ipython-input-17-426901edcb81> in <module>
----> 1 import tweepy
      2 from tweepy import API
      3 from tweepy import OAuthHandler
      4
      5 CONSUMER_KEY = XXXXX
```

```
ModuleNotFoundError: No module named 'tweepy'
```

1.2.5 1.5 Temps

In [18]: *#Timeline dels twits publicats sobre la controvèrsia generada pel programa La Gent No*

```
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime as dt

df = pd.read_csv("lagentnormal-CORPUS.csv", sep='\t', encoding='utf-8')

df1 = df[['Time', 'User']]

df1['Time'] = pd.to_datetime(df1['Time']).dt.date #Treure hores i minuts

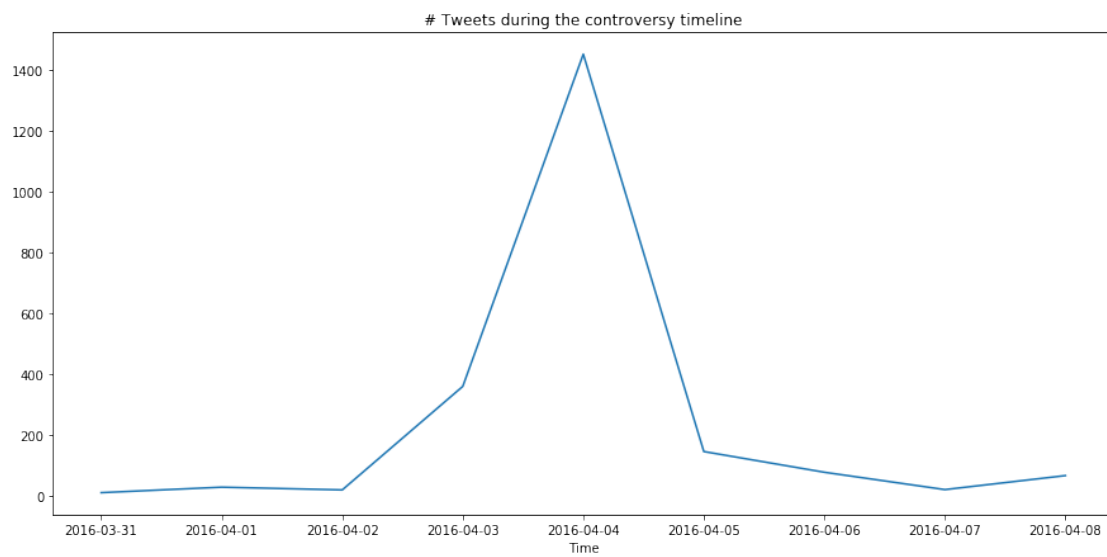
df1 = df1.groupby('Time').Time.count()
print(df1)

fig, ax = plt.subplots(figsize=(15,7))
df1.plot(title='# Tweets during the controversy timeline')
plt.show()
```

```
/Library/Frameworks/Python.framework/Versions/3.6/lib/python3.6/site-packages/ipykernel_launcher
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: <http://pandas.pydata.org/pandas-docs/stable/indexing.html>
 # This is added back by InteractiveShellApp.init_path()

```
Time
2016-03-31      10
2016-04-01      28
2016-04-02      19
2016-04-03     359
2016-04-04    1450
2016-04-05     145
2016-04-06      77
2016-04-07      20
2016-04-08      66
Name: Time, dtype: int64
```



```
In [ ]:
```